SUPERMERCADO DB

Guía de Estudio Completa

React + Vite + React Router · Preparación para Entrevista Técnica

📌 Este documento cubre desde cero los fundamentos de React hasta el análisis línea por línea de cada archivo del proyecto. Diseñado para que puedas defender tu prueba técnica con confianza total.

PARTE 1 · Fundamentos de React

1.1 ¿Qué es React?

React es una biblioteca de JavaScript creada por Meta (Facebook) para construir interfaces de usuario. Su idea central es dividir la UI en piezas reutilizables llamadas componentes.

Concepto	Significado
`Librería, no framework`	React solo maneja la vista (la V en MVC). Para routing, estado global, etc., se agregan librerías como React Router.
`Virtual DOM`	React mantiene una copia virtual del DOM en memoria. Cuando algo cambia, compara el virtual con el real y solo actualiza las partes necesarias. Esto lo hace muy rápido.
`Declarativo`	Tú describes cómo debe verse la UI; React se encarga de actualizar el DOM para que coincida.
`Unidireccional`	Los datos fluyen de padre a hijo (props). Para comunicar hacia arriba, se pasan funciones como props.

1.2 JSX — JavaScript + XML

JSX es la sintaxis especial que permite escribir HTML dentro de JavaScript. Vite lo transforma en llamadas a React.createElement() antes de ejecutarse en el navegador.

```
// JSX
const elemento = <h1 className="titulo">Hola mundo</h1>;

// Lo que Vite/Babel convierte internamente:
const elemento = React.createElement('h1', { className: 'titulo' }, 'Hola mundo');
```

💡 *Reglas clave de JSX: (1) Siempre debe haber un solo elemento raíz (usa <> </> si no quieres un div extra). (2) Los atributos usan camelCase: class → className, for → htmlFor. (3) Las expresiones JS van entre llaves { }.*

1.3 Componentes

Un componente es una función (o clase) que recibe datos y retorna JSX. En este proyecto usamos únicamente componentes funcionales, que es el estándar moderno.

```
// Componente funcional básico
function Saludo({ nombre }) {        // recibe props como argumento
  return <h1>Hola, {nombre}!</h1>;   // retorna JSX
}

// Uso:
<Saludo nombre="María" />
```

Término	Descripción
Props	Datos que el padre pasa al hijo. Son de solo lectura — el hijo nunca debe modificarlas.
State	Datos internos del componente que pueden cambiar con el tiempo. Al cambiar, React vuelve a renderizar el componente.
Render	Cada vez que el estado o las props cambian, React ejecuta la función del componente de nuevo y actualiza el DOM.

1.4 Hooks principales

Los Hooks son funciones especiales que permiten usar características de React (estado, ciclo de vida, etc.) dentro de componentes funcionales.

useState

Crea una variable de estado. Retorna el valor actual y una función para actualizarlo.

```
const [count, setCount] = useState(0);  // valor inicial: 0

// Para actualizar:
setCount(5);
setCount(prev => prev + 1);  // forma funcional (más segura)
```

useEffect

Ejecuta código con efectos secundarios: llamadas a API, suscripciones, etc. Se ejecuta después de que el componente se renderiza.

```
useEffect(() => {
  // Se ejecuta al montar el componente (array vacío [])
  fetchData();
}, []);  // <-- array de dependencias
```

```
useEffect(() => {
  // Se ejecuta cada vez que 'id' cambia
  cargarProducto(id);
}, [id]);
```

📌 Array de dependencias en useEffect:
[] → corre solo una vez al montar el componente
[id] → corre al montar Y cada vez que 'id' cambie
sin array → corre en CADA render (¡peligroso!)

1.5 ¿Qué es Vite?

Vite es una herramienta de desarrollo moderna que reemplaza a Create React App (CRA). Es mucho más rápida porque usa ES Modules nativos del navegador en desarrollo, en vez de empaquetar todo antes.

Comando/Archivo	Para qué sirve
`npm run dev`	Inicia el servidor de desarrollo en localhost:5173
`npm run build`	Genera la carpeta dist/ lista para producción
`npm run preview`	Previsualiza el build de producción localmente
`vite.config.js`	Archivo de configuración de Vite (plugins, alias, proxy, etc.)
`index.html`	En Vite está en la RAÍZ del proyecto, no en /public como en CRA

PARTE 2 · React Router v6

2.1 ¿Por qué React Router?

React por sí solo no tiene navegación entre páginas. React Router añade esa capacidad, permitiendo que la URL cambie sin recargar el navegador (SPA — Single Page Application).

Componente/Hook	Función
`BrowserRouter`	Envuelve toda la app. Habilita el historial del navegador (URL reales).
`Routes`	Contenedor de todas las rutas. Solo renderiza la que coincida.
`Route`	Define qué componente mostrar para cada path.
`Link`	Como un <a>, pero sin recargar la página.
`NavLink`	Igual que Link pero agrega clase 'active' automáticamente cuando la ruta coincide.
`useNavigate`	Hook para navegar programáticamente (ej: después de un submit).
`useParams`	Hook para leer parámetros de la URL como :id.

```
// Ejemplo completo de configuración (App.jsx del proyecto)
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/productos" element={<IndexProductos />} />
    <Route path="/productos/editar/:id" element={<EditarProducto />} />
  </Routes>
</BrowserRouter>
```

¿Cómo leer el :id en EditarProducto?

```
import { useParams } from 'react-router-dom';

function EditarProducto() {
  const { id } = useParams();  // lee el :id de la URL
  // Si la URL es /productos/editar/42, entonces id = '42'
}
```

¿Cómo navegar después de guardar un formulario?

```
import { useNavigate } from 'react-router-dom';

function CrearProducto() {
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    await fetch('/productos', { method: 'POST', ... });
    navigate('/productos');  // redirige después de guardar
  }
}
```

PARTE 3 · Estructura del Proyecto

3.1 Árbol de archivos

```
supermercado-vite/
│
├── index.html              ← punto de entrada HTML (raíz en Vite)
├── vite.config.js          ← configuración de Vite
├── package.json            ← dependencias y scripts
│
└── src/
    ├── main.jsx            ← punto de entrada JavaScript
    ├── App.jsx             ← router principal
    ├── index.css           ← estilos globales
    ├── api.js              ← URL base del backend
    │
    ├── components/         ← piezas reutilizables
    │   ├── Navbar.jsx
    │   ├── ConfirmDialog.jsx
    │   └── ResultBox.jsx
    │
    └── pages/              ← una página por ruta
        ├── Home.jsx
        ├── productos/
        │   ├── IndexProductos.jsx
        │   ├── CrearProducto.jsx
        │   └── EditarProducto.jsx
        └── inventario/
            ├── IndexInventario.jsx
            ├── CrearInventario.jsx
            └── EditarInventario.jsx
```

3.2 Convención de carpetas

Carpeta/Archivo	Responsabilidad
`components/`	Componentes reutilizables que NO son páginas completas. Se usan en múltiples lugares.
`pages/`	Un archivo por cada ruta visible. Cada página puede usar componentes de /components.
`api.js`	Centraliza la URL del backend. Si el puerto cambia, solo se edita aquí.
`main.jsx`	Solo monta la app en el DOM. No tiene lógica de negocio.
`App.jsx`	Solo define las rutas. No tiene lógica de negocio.
`index.css`	Estilos globales accesibles desde cualquier componente.

PARTE 4 · Análisis Línea por Línea de Cada Archivo

4.1 index.html (raíz del proyecto)

📌 *En Vite, este archivo vive en la raíz del proyecto, no en /public. Es la diferencia más importante respecto a Create React App.*

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Supermercado DB</title>
    <!-- Fuentes de Google Fonts (DM Serif, DM Sans, DM Mono) -->
    <link href="https://fonts.googleapis.com/..." rel="stylesheet"/>
  </head>
  <body>
    <div id="root"></div>   <!-- React inyecta toda la app aquí -->
    <script type="module" src="/src/main.jsx"></script>  <!-- punto de entrada -->
  </body>
</html>
```

- **id="root"**: el div vacío donde React monta toda la aplicación.
- **type="module"**: Vite usa ES Modules nativos. Sin esto, los imports no funcionarían.
- La fuente se carga desde Google Fonts porque el CSS la referencia con `font-family: 'DM Sans'`.

4.2 vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],   // habilita JSX, Fast Refresh, etc.
})
```

@vitejs/plugin-react hace dos cosas importantes:

- Transforma JSX a JavaScript puro (React.createElement) durante el build.
- Habilita Fast Refresh: los cambios en el código se reflejan en el navegador en milisegundos sin perder el estado.

4.3 package.json

```
{
  "type": "module",           // el proyecto usa import/export (ES Modules)
  "scripts": {
    "dev":     "vite",          // npm run dev  → servidor de desarrollo
    "build":   "vite build",    // npm run build → genera /dist
    "preview": "vite preview"   // npm run preview → prueba el /dist local
  },
  "dependencies": {            // se incluyen en producción
    "react":            "^18",
    "react-dom":        "^18",
    "react-router-dom": "^6"
  },
  "devDependencies": {         // solo para desarrollo
    "@vitejs/plugin-react": "^4",
    "vite": "^5"
  }
}
```

💡 dependencies vs devDependencies:
dependencies → código que se envía al navegador (React, Router)
devDependencies → herramientas solo para el desarrollador (Vite, linters)

4.4 src/api.js

```
export const API_BASE = 'http://localhost:9874';
```

Aunque parece trivial, este archivo es muy importante por el principio DRY (Don't Repeat Yourself). Si el puerto del backend cambia, se edita en UN solo lugar en vez de buscar y reemplazar en todos los archivos.

- Cada página lo importa así: `import { API_BASE } from '../../api';`
- Y lo usa: `fetch(`${API_BASE}/productos`)`

4.5 src/main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'           // estilos globales
```

```
ReactDOM.createRoot(document.getElementById('root'))  // busca el <div id='root'>
  .render(
    <React.StrictMode>    // modo estricto: detecta errores en desarrollo
      <App />             // monta toda la app
    </React.StrictMode>
  )
```

- **ReactDOM.createRoot**: API moderna de React 18 para montar la app.
- **React.StrictMode**: en desarrollo, renderiza los componentes dos veces para detectar efectos secundarios inesperados. No afecta producción.
- El `import './index.css'` aquí garantiza que los estilos globales estén disponibles en toda la app.

4.6 src/App.jsx

```
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
// ... imports de páginas

export default function App() {
  return (
    <BrowserRouter>           {/* habilita navegación sin recarga */}
      <Navbar />              {/* siempre visible, fuera de Routes */}
      <Routes>               {/* solo renderiza la ruta que coincide */}
        <Route path="/" element={<Home />} />
        <Route path="/productos" element={<IndexProductos />} />
        <Route path="/productos/crear" element={<CrearProducto />} />
        <Route path="/productos/editar/:id" element={<EditarProducto />} />
        <Route path="/inventario" element={<IndexInventario />} />
        <Route path="/inventario/crear" element={<CrearInventario />} />
        <Route path="/inventario/editar" element={<EditarInventario />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Puntos clave de este archivo:

- **<Navbar /> está FUERA de <Routes>**: esto significa que el Navbar se muestra en TODAS las páginas, independientemente de la ruta activa.
- **<Routes> renderiza solo un hijo**: busca la ruta que coincida con la URL y renderiza solo ese componente.
- **:id en la ruta**: el ":" indica un parámetro dinámico. La URL /productos/editar/42 pone 42 disponible con useParams().

PARTE 5 · Componentes Reutilizables (components/)

5.1 Navbar.jsx

> *Barra de navegación que aparece en todas las páginas.*

```
import { NavLink } from 'react-router-dom';

export default function Navbar() {
  return (
    <nav className="top-nav">
      <NavLink to="/" className="brand">
        Supermercado <span>DB</span>
      </NavLink>
      <div className="top-nav-links">
        <NavLink to="/" end className={({ isActive }) => isActive ? 'active' :
''}>
          Inicio
        </NavLink>
        <NavLink to="/productos" className={({ isActive }) => isActive ? 'active'
: ''}>
          Productos
        </NavLink>
        <NavLink to="/inventario" className={({ isActive }) => isActive ? 'active'
: ''}>
          Inventario
        </NavLink>
      </div>
    </nav>
  );
}
```

Conceptos clave en este componente:

- **NavLink vs Link**: NavLink añade automáticamente una clase 'active' cuando la ruta coincide. Perfecto para navbars.
- **className como función**: NavLink pasa { isActive } a la función. Retornamos 'active' o '' según corresponda.
- **prop end en <NavLink to="/">**: sin 'end', el link de Inicio estaría activo en TODAS las rutas porque todas empiezan con '/'. 'end' lo limita a la coincidencia exacta.

5.2 ConfirmDialog.jsx

> *Modal de confirmación reutilizable para operaciones de eliminación.*

```
export default function ConfirmDialog({ open, message, onConfirm, onCancel }) {
  return (
    <div className={`confirm-overlay ${open ? 'open' : ''}`}>
      <div className="confirm-box">
        <h6>Confirmar eliminación</h6>
        <p>{message}</p>
        <div className="confirm-actions">
          <button className="btn-secondary" onClick={onCancel}>Cancelar</button>
          <button className="btn-danger" onClick={onConfirm}>Sí, eliminar</button>
        </div>
      </div>
    </div>
  );
}
```

Props que recibe:

Prop	Descripción
`open`	boolean — si es true, el CSS aplica clase 'open' que hace visible el overlay.
`message`	string — el texto de la pregunta ('¿Deseas eliminar...?').
`onConfirm`	función — se ejecuta al hacer clic en 'Sí, eliminar'.
`onCancel`	función — se ejecuta al hacer clic en 'Cancelar'.

- **Template literal con condición**: `` `confirm-overlay ${open ? 'open' : ''}` `` añade la clase 'open' dinámicamente.
- **El padre controla el estado**: este componente no tiene estado propio. IndexProductos.jsx maneja open/closed con useState.
- **Patrón de elevación de estado**: las funciones onConfirm/onCancel viven en el padre porque el padre es quien sabe qué eliminar.

5.3 ResultBox.jsx

> *Caja de retroalimentación que muestra mensajes de éxito o error tras una operación.*

```
export default function ResultBox({ message, isError }) {
  if (!message) return null;   // si no hay mensaje, no renderiza nada
  return (
    <div className={`result-box ${isError ? 'result-error' : 'result-ok'}`}>
      {message}
    </div>
  );
}
```

- **if (!message) return null**: retornar null en React es perfectamente válido. El componente no renderiza nada hasta tener un mensaje.
- **isError**: booleano que selecciona entre los estilos CSS 'result-error' (fondo rojo) o 'result-ok' (fondo verde).
- Este patrón evita la lógica de `display: none` que teníamos en el HTML/JS original, haciendo el código más limpio.

PARTE 6 · Páginas de Productos (pages/productos/)

6.1 IndexProductos.jsx — Listar con eliminación

> *GET /productos · Muestra la tabla de productos y gestiona la eliminación con modal de confirmación.*

Estado del componente

```
const [productos, setProductos] = useState([]);
// ^ lista vacía inicial; se llena con los datos de la API

const [status, setStatus]     = useState('Cargando…');
// ^ texto informativo en la toolbar (ej: '5 productos')

const [error, setError]       = useState(false);
// ^ true si el fetch falló; muestra fila de error en la tabla

const [confirm, setConfirm]   = useState({ open: false, id: null, nombre: '' });
// ^ estado del modal: open=visible, id=qué eliminar, nombre=para el mensaje
```

Flujo de datos

1. El componente se monta → useEffect llama a listar().
2. listar() hace `fetch('http://localhost:9874/productos')`.
3. Si OK: setProductos(data) → React re-renderiza → aparecen las filas.
4. Si error: setError(true) → aparece fila roja de error.
5. Usuario hace clic en Eliminar → abrirConfirm(id, nombre) → modal visible.
6. Usuario confirma → eliminar() → DELETE a la API → listar() de nuevo.

Código del fetch y renderizado

```
useEffect(() => { listar(); }, []);  // solo al montar

async function listar() {
  try {
    const res = await fetch(`${API_BASE}/productos`);
    if (!res.ok) throw new Error();     // cualquier 4xx/5xx lanza el catch
    const data = await res.json();
    setProductos(data);
    setStatus(`${data.length} productos`);
  } catch {
    setError(true);
    setStatus('Error');
```

```
  }
}

// En el JSX, renderizado condicional de la tabla:
{error ? (
  <tr><td colSpan='7' className='error'>Error al cargar</td></tr>
) : productos.length === 0 ? (
  <tr><td colSpan='7'>No hay productos</td></tr>
) : (
  productos.map(p => (         // .map() genera una fila por producto
    <tr key={p.id_producto}>   // key es obligatorio en listas
      <td>{p.id_producto}</td>
      ...
    </tr>
  ))
)}
```

⚠️ prop key en listas: Cuando usas .map() para renderizar una lista en React, cada elemento debe tener una prop key única y estable (no el índice del array). React la usa internamente para identificar qué elementos cambiaron. Sin key, React lanza una advertencia y el rendimiento puede degradarse.

6.2 CrearProducto.jsx — Formulario POST

> *POST /productos · Formulario controlado para crear un nuevo producto.*

Formulario controlado (Controlled Component)

En React, un formulario controlado es aquel donde el estado de React es la 'única fuente de verdad' para los inputs. Cada cambio en el input actualiza el estado, y el estado determina el valor del input.

```
const [form, setForm] = useState({
  codigo: '', nombre: '', categoria: '', sub_categoria: '', marca: ''
});

// Un solo handler para todos los inputs:
function handleChange(e) {
  setForm(prev => ({ ...prev, [e.target.id]: e.target.value }));
  //                            ^ computed property name: usa el id del input
  //              ^ spread operator: copia el objeto anterior
}

// El input queda enlazado:
```

```
<input id="codigo" value={form.codigo} onChange={handleChange} />
//                   ^ React controla el valor  ^ React actualiza el estado
```

Submit y llamada a la API

```
async function handleSubmit(e) {
  e.preventDefault();   // evita que el browser recargue la página

  // Validación antes de enviar:
  if (!form.codigo || !form.nombre /* ... */) {
    setResult({ message: 'Todos los campos son obligatorios', isError: true });
    return;
  }

  try {
    const res = await fetch(`${API_BASE}/productos`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ codigo_producto: form.codigo, ... })
    });
    if (!res.ok) throw new Error();
    setResult({ message: 'Producto creado correctamente', isError: false });
    setTimeout(() => navigate('/productos'), 1500);  // redirige en 1.5s
  } catch {
    setResult({ message: 'Error al crear producto', isError: true });
  }
}
```

6.3 EditarProducto.jsx — Cargar y actualizar (GET + PUT)

> *GET /productos/:id para cargar · PUT /productos/:id para actualizar.*

Este es el componente más complejo porque hace DOS llamadas a la API:

7. Al montar: GET para cargar los datos actuales del producto.
8. Al hacer submit: PUT para enviar los datos actualizados.

```
const { id } = useParams();   // lee el :id de la URL
const navigate = useNavigate();
const [loading, setLoading]   = useState(true);   // mientras carga
const [loadError, setLoadError] = useState(false); // si falla la carga

useEffect(() => {
  async function cargar() {
    try {
```

```
      const res = await fetch(`${API_BASE}/productos/${id}`);
      if (!res.ok) throw new Error();
      const data = await res.json();
      setForm({               // rellena el formulario con los datos existentes
        codigo: data.codigo_producto,
        nombre: data.nombre_producto,
        // ...
      });
      setLoading(false);
    } catch {
      setLoadError(true);
      setLoading(false);
    }
  }
  cargar();
}, [id]);  // si el id cambia, vuelve a cargar

// Renderizado condicional:
{loading && <p>Cargando datos del producto…</p>}
{loadError && <p>Error al cargar producto.</p>}
{!loading && !loadError && <form>...</form>}
```

PARTE 7 · Páginas de Inventario (pages/inventario/)

7.1 IndexInventario.jsx — Existencias totales

> *GET /inventario → llama al stored procedure sp_inventario_existencias() en el backend.*

Este componente es más simple que IndexProductos porque no tiene eliminación. Solo lista las existencias totales agrupadas por producto.

```
// El SP puede retornar los campos con diferentes nombres según implementación.
// Por eso usamos el operador '??' (nullish coalescing) para manejar ambos casos:

const codigo    = r.codigo_producto ?? r.codigo ?? '—';
const nombre    = r.nombre_producto ?? r.nombre ?? '—';
const existencia = r.existencia_total ?? r.total ?? r.existencia ?? 0;

// a ?? b significa: si 'a' es null o undefined, usa 'b'
```

💡 Operador ?? (nullish coalescing) vs ||:
a ?? b → usa b solo si a es null o undefined
a || b → usa b si a es cualquier valor falsy (0, '', false, null, undefined)

Ejemplo: si existencia_total = 0, con || daría '—' (incorrecto), con ?? da 0 (correcto).

7.2 CrearInventario.jsx — Validaciones numéricas

> *POST /inventario · Similar a CrearProducto pero con validaciones de tipos numéricos.*

```
// Conversión y validación de campos numéricos:
const payload = {
  cantidad: Number(form.cantidad),  // string del input → número
  costo:    Number(form.costo),
};

// Number.isFinite() valida que sea un número real (no NaN, no Infinity):
if (!Number.isFinite(payload.cantidad) || payload.cantidad <= 0) {
  setResult({ message: 'La cantidad debe ser mayor a 0', isError: true });
  return;
}
```

```
// Para campos opcionales, se envía null si están vacíos:
fecha_vencimiento: form.fecha_vencimiento || null
```

Expresión	Resultado
`Number('42')`	→ 42 (número válido)
`Number('')`	→ 0 (OJO: string vacío se convierte a 0, no a NaN)
`Number('abc')`	→ NaN (no es un número)
`Number.isFinite(NaN)`	→ false (protege contra NaN)
`Number.isFinite(Infinity)`	→ false (protege contra Infinity)
`Number.isFinite(42)`	→ true (válido)

7.3 EditarInventario.jsx — Edición por código

> *PUT /inventario/codigo/:codigo · El usuario ingresa el código manualmente (no viene de la URL).*

Este componente tiene una diferencia importante con EditarProducto: el ID no viene de la URL sino que el usuario lo escribe en el formulario. Por eso no usa useParams() y no hace GET previo.

```
// La URL es simplemente /inventario/editar (sin :id)
// El código del producto es un campo más del formulario.

const res = await fetch(
  `${API_BASE}/inventario/codigo/${encodeURIComponent(codigoProducto)}`,
  { method: 'PUT', ... }
);

// encodeURIComponent() es importante si el código tiene caracteres especiales
// como espacios o guiones que podrían romper la URL.
```

PARTE 8 · Sistema de Estilos (index.css)

8.1 Variables CSS (Design Tokens)

```
:root {
  --bg:      #f5f2eb;  /* fondo general (crema) */
  --surface: #ffffff;  /* tarjetas y paneles */
  --ink:     #1a1a18;  /* texto principal */
  --muted:   #7a7a72;  /* texto secundario */
  --accent:  #c84b2f;  /* rojo-naranja para acción */
  --border:  #e0ddd6;  /* bordes sutiles */
  --nav-bg:  #1a1a18;  /* fondo del navbar */
}
```

Las variables CSS (custom properties) permiten cambiar el tema completo en un solo lugar. En React, es equivalente a tener un objeto de tema. Si el cliente quiere cambiar el color de acento, solo se edita --accent.

8.2 Clases importantes del proyecto

Clase CSS	Propósito
`.top-nav`	Barra de navegación oscura sticky (se queda arriba al hacer scroll).
`.page-header`	Zona del título y breadcrumb de cada página. Max-width 960px centrado.
`.content-area`	Contenedor principal de cada página. Max-width 960px centrado.
`.card-panel`	Tarjeta blanca con borde y padding donde vive el contenido.
`.form-grid`	CSS Grid de 2 columnas para los formularios. 1 columna en móvil.
`.toolbar`	Flexbox: status a la izquierda, botones a la derecha.
`.btn-primary`	Botón rojo sólido para acción principal.
`.btn-secondary`	Botón con borde para acciones secundarias (Cancelar, Editar).
`.btn-danger`	Botón rojo outline para eliminar.
`.result-box`	Caja de mensaje de éxito (verde) o error (rojo).
`.confirm-overlay`	Overlay oscuro semitransparente. Clase 'open' lo hace visible con CSS transition.
`.method-badge`	Pequeña etiqueta coloreada: POST (azul), PUT (amarillo), GET (verde).

`.breadcrumb-tag`	Texto pequeño en monospace que muestra la ruta de navegación.

PARTE 9 · Flujos Completos de Datos

9.1 Flujo: Crear un producto

```
Usuario llena el formulario
       ↓
handleChange() actualiza form en useState
       ↓
Usuario hace clic en 'Guardar'
       ↓
handleSubmit() → e.preventDefault()
       ↓
Validación de campos → si falla: setResult(error) → STOP
       ↓
fetch POST /productos con JSON body
       ↓
  ┌── res.ok? ──────────────────────────┐
  │ Sí                                  │ No
  ↓                                     ↓
setResult('Producto creado')        setResult('Error...')
  ↓
setTimeout 1500ms → navigate('/productos')
```

9.2 Flujo: Eliminar un producto

```
Usuario hace clic en botón 'Eliminar' de una fila
       ↓
abrirConfirm(id, nombre) → setConfirm({ open: true, id, nombre })
       ↓
ConfirmDialog recibe open=true → se hace visible con CSS
       ↓
  ┌── Usuario elige ────────────────────┐
  │ 'Cancelar'                          │ 'Sí, eliminar'
  ↓                                     ↓
setConfirm({ open: false })         fetch DELETE /productos/:id
                                         ↓
                                    listar()  ← refresca la tabla
```

9.3 Flujo: Editar un producto

```
Usuario navega a /productos/editar/42
       ↓
EditarProducto se monta → useParams() lee id='42'
       ↓
```

```
useEffect → cargar(42) → fetch GET /productos/42
        ↓
setForm({ codigo: ..., nombre: ..., ... })  ← llena el formulario
setLoading(false) → formulario se hace visible
        ↓
Usuario edita campos → handleChange actualiza form
        ↓
Submit → fetch PUT /productos/42 con datos nuevos
        ↓
navigate('/productos')
```

PARTE 10 · Preguntas Frecuentes de Entrevista

10.1 Preguntas sobre React

¿Qué es el Virtual DOM?

Es una representación en memoria del DOM real. Cuando el estado cambia, React crea un nuevo Virtual DOM, lo compara con el anterior (proceso llamado diffing o reconciliación) y solo actualiza las partes del DOM real que cambiaron. Esto es mucho más eficiente que actualizar todo el DOM.

¿Cuál es la diferencia entre props y state?

Las props son datos que vienen del padre y son de solo lectura — el componente hijo no puede modificarlas. El state es datos internos del componente que pueden cambiar con el tiempo usando setState (o el setter de useState). Cuando el state cambia, React re-renderiza el componente.

¿Por qué se necesita la prop key en las listas?

React necesita la key para identificar de forma eficiente qué elementos de una lista cambiaron, se agregaron o se eliminaron. Sin key única, React podría re-renderizar incorrectamente o mostrar datos mezclados. Nunca se debe usar el índice del array como key cuando la lista puede reordenarse.

¿Qué hace exactamente useEffect?

useEffect permite ejecutar código con efectos secundarios (llamadas a APIs, suscripciones, manipulación directa del DOM) después de que el componente se renderiza. El array de dependencias controla cuándo se ejecuta: [] = solo al montar, [x] = al montar y cuando x cambie, sin array = en cada render.

¿Qué es un componente controlado?

Un componente controlado es un input cuyo valor está completamente controlado por el estado de React. El valor del input viene del state (value={form.campo}) y cada cambio actualiza el state (onChange={handleChange}). React se convierte en la única fuente de verdad del formulario.

¿Por qué se usa async/await con fetch?

fetch() retorna una Promise. Con async/await el código asíncrono se puede escribir de forma secuencial y legible, como si fuera síncrono. await pausa la ejecución de la función async hasta que la Promise resuelve. El try/catch captura errores de red o de la API.

10.2 Preguntas sobre el proyecto

¿Por qué separaste componentes y páginas en carpetas diferentes?

Porque tienen responsabilidades distintas. Las páginas representan rutas completas de la aplicación: cada una corresponde a una URL. Los componentes son piezas reutilizables que se usan en múltiples páginas (Navbar, ResultBox, ConfirmDialog). Esta separación facilita el mantenimiento y la escalabilidad.

¿Para qué sirve api.js?

Centraliza la URL base del backend. Si el servidor cambia de puerto o de dirección, solo se modifica en un archivo en vez de buscar y reemplazar en todos los archivos del proyecto. Es el principio DRY (Don't Repeat Yourself) aplicado a la configuración.

¿Por qué ConfirmDialog no tiene estado propio?

Porque no necesita saber QUÉ va a eliminar ni manejar la lógica de la API. El estado (open, id, nombre) vive en el componente padre (IndexProductos) que sí tiene esa información. ConfirmDialog solo se encarga de la presentación visual y notifica al padre cuándo el usuario confirmó o canceló. Este patrón se llama elevación de estado (state lifting).

¿Cuál es la diferencia entre Vite y Create React App?

Vite es significativamente más rápido en desarrollo porque sirve los módulos nativamente con ES Modules sin hacer un bundle completo. CRA empaqueta todo con Webpack antes de servirlo, lo que lo hace lento para proyectos grandes. Vite también tiene configuración más simple y genera builds de producción más optimizados. La diferencia práctica: Vite arranca en ~1 segundo, CRA puede tomar 10-30 segundos.

¿Cómo le pasarías datos de un componente hijo al padre?

En React los datos fluyen de padre a hijo (unidireccional). Para comunicar del hijo al padre, el padre le pasa una función como prop. El hijo llama esa función con los datos. Por ejemplo, ConfirmDialog recibe onConfirm como prop y la llama cuando el usuario confirma. El padre decide qué hacer con esa notificación.

¿Qué harías si la API tarda mucho en responder?

Agregaría un estado de loading (useState(false)) que se activa antes del fetch y se desactiva al terminar. En el JSX mostraría un spinner o deshabilitar el botón de submit mientras loading es true para evitar envíos duplicados. También consideraría agregar un timeout al fetch o manejar el caso de cancelación con AbortController.

PARTE 11 · Comparativa: HTML/JS Vanilla vs React

11.1 Tabla de equivalencias

HTML/JS Vanilla	React + Vite
`window.location.href = 'otra.html'`	`navigate('/otra-ruta')` via `useNavigate()`
`document.getElementById('input').value`	`form.campo` via `useState` + `value={}`
`input.addEventListener('change', fn)`	`onChange={handleChange}` en el JSX
`document.querySelector('#tbody').innerHTML = ...`	`setProductos(data)` → React re-renderiza
`<script type='module' src='listar.js'>`	Componente ListarProductos con useEffect
Múltiples archivos HTML (una URL = un archivo)	Una SPA con React Router (una URL = una ruta)
`?id=42` en la URL leído con `URLSearchParams`	`:id` en la ruta leído con `useParams()`
`display: none` / `display: block` con JS	Renderizado condicional: `{condition && <comp>}`
`setTimeout(() => location.href = ..., 1500)`	`setTimeout(() => navigate('/ruta'), 1500)`

11.2 Por qué React es mejor para este proyecto

- Estado centralizado: en vez de manipular el DOM manualmente, describes cómo debe verse la UI según los datos. React se encarga de actualizar el DOM.
- Componentes reutilizables: ConfirmDialog y ResultBox se usan en múltiples páginas sin duplicar código.
- Navegación sin recarga: React Router cambia la URL y el contenido sin recargar, lo que da una experiencia más fluida.
- Escalabilidad: agregar una nueva página es tan simple como crear un componente y añadir una Route. En HTML vanilla requeriría crear un nuevo archivo HTML + JS.
- Mantenimiento: cuando cambia la URL del backend, se edita api.js. En vanilla había que actualizar cada archivo .js por separado.

PARTE 12 · Glosario Rápido

Término	Definición
`SPA`	Single Page Application. Aplicación de una sola página que cambia el contenido dinámicamente sin recargar el navegador.
`Component`	Función JavaScript que retorna JSX. Es la unidad básica de construcción de UIs en React.
`Props`	Propiedades que el padre pasa al hijo. Son inmutables desde el punto de vista del hijo.
`State`	Datos internos de un componente. Al cambiar con setState/setter, React re-renderiza el componente.
`Hook`	Función especial de React que empieza con 'use'. Permite usar estado y ciclo de vida en componentes funcionales.
`useState`	Hook para declarar variables de estado en un componente.
`useEffect`	Hook para ejecutar efectos secundarios (fetch, suscripciones) después del renderizado.
`useParams`	Hook de React Router para leer parámetros dinámicos de la URL (:id).
`useNavigate`	Hook de React Router para navegar programáticamente entre rutas.
`JSX`	Extensión de sintaxis que permite escribir HTML dentro de JavaScript. Vite lo transforma en JS puro.
`Virtual DOM`	Copia en memoria del DOM real. React la usa para calcular el mínimo número de actualizaciones necesarias.
`Reconciliación`	Proceso por el que React compara el Virtual DOM anterior con el nuevo para determinar qué cambió.
`Rendering condicional`	Mostrar u ocultar elementos según una condición: {cond && <Comp>} o {cond ? <A> : <B>}.
`Spread operator`	{ ...obj, key: val } crea una copia del objeto con una propiedad modificada. Esencial para actualizar state de objetos.
`DRY`	Don't Repeat Yourself. Principio que dice que la lógica debe estar en un solo lugar (ej: api.js).
`ESM`	ES Modules. Sistema de módulos nativo del navegador moderno. Vite lo aprovecha para un dev server ultra rápido.
`Async/Await`	Sintaxis para manejar Promises de forma secuencial y legible. Siempre va con try/catch para manejar errores.
`REST API`	Estilo de arquitectura para APIs HTTP. Usa verbos (GET, POST, PUT, DELETE) y URLs como recursos.

`JSON.stringify`	Convierte un objeto JavaScript a texto JSON para enviarlo en el body de un fetch.
`encodeURIComponent`	Codifica caracteres especiales en una URL para que no rompan la petición HTTP.

¡Mucho éxito en tu entrevista!

Supermercado DB · React + Vite · Guía de Estudio